

TUGAS 2
MATA KULIAH
Struktur Data / STSI4104.104



UNIVERSITAS TERBUKA

DISUSUN OLEH

NAMA : Yudo Sandewo
NIM : 052968524
KODE KELAS : 104

PROGRAM STUDI ILMU SISTEM INFORMASI

Fakultas Sains dan Teknologi

UPBJJ UT JAKARTA

UNIVERSITAS TERBUKA

Nama: Yudosandewo

NIM: 052968524

Prodi: Sistem Informasi

Source Code :

```
public class Soal1 {

    /*
     * ANALISA ALGORITMA MERGE SORT:
     * -----
     * - Merge Sort bekerja dengan konsep "Divide and Conquer".
     * - Array dibagi menjadi dua bagian berulang-ulang sampai ukuran 1 elemen.
     * - Setiap bagian kecil digabung kembali dalam keadaan sudah terurut.
     * - Waktu eksekusi stabil:  $O(n \log n)$  pada best, average, worst case.
     * - Cocok untuk dataset besar.
     * - Pada tugas ini, sorting dilakukan dari besar  $\rightarrow$  kecil (descending).
     */

    // Fungsi utama Merge Sort
    public static void mergeSort(int[] arr, int left, int right) {

        // Proses berhenti jika hanya 1 elemen
        if (left < right) {

            // Titik tengah untuk membagi array
            int mid = (left + right) / 2;

            // MEMBAGI BAGIAN KIRI
            mergeSort(arr, left, mid);

            // MEMBAGI BAGIAN KANAN
            mergeSort(arr, mid + 1, right);

            // MENGGABUNGKAN BAGIAN KIRI & KANAN YANG SUDAH TERURUT
            merge(arr, left, mid, right);
        }
    }

    // Fungsi merge untuk menggabungkan dua sub-array
    public static void merge(int[] arr, int left, int mid, int right) {

        // Panjang dua sub-array
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Menyimpan bagian kiri & kanan
        int[] L = new int[n1];
        int[] R = new int[n2];

        // Copy data ke array sementara
        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for (int i = 0; i < n2; i++)
            R[i] = arr[mid + 1 + i];

        int i = 0, j = 0, k = left;
```

```

/*
 * PROSES PENGABUNGAN (MERGING)
 * -----
 * - Di sini dilakukan perbandingan elemen kiri vs kanan
 * - Karena descending, kita memilih elemen yang lebih besar terlebih dahulu
 * - Inilah inti dari algoritma merge sort
 */
while (i < n1 && j < n2) {
    if (L[i] >= R[j]) {
        arr[k++] = L[i++];
    } else {
        arr[k++] = R[j++];
    }
}

// Jika masih ada sisa elemen kiri
while (i < n1)
    arr[k++] = L[i++];

// Jika masih ada sisa elemen kanan
while (j < n2)
    arr[k++] = R[j++];
}

public static void printArr(int[] arr) {
    for (int i : arr)
        System.out.print(i + " ");
    System.out.println();
}

public static void main(String[] args) {

    // DATA AWAL (Belum terurut)
    int[] data = { 42, 17, 8, 60, 23, 91, 15, 50 };

    System.out.println("=== MERGE SORT DESCENDING ===");
    System.out.print("Data sebelum sorting: ");
    printArr(data);

    // Memulai proses merge sort
    mergeSort(data, 0, data.length - 1);

    System.out.print("Data setelah sorting: ");
    printArr(data);

    /*
     * ANALISA KINERJA MERGE SORT:
     * -----
     * - Waktu:  $O(n \log n)$ 
     * - (karena proses divide  $\log n$ , merge  $n$ )
     * - Ruang:  $O(n)$  → membutuhkan array tambahan.
     * - Kelebihan:
     * - Stabil dan konsisten dalam performa
     * - Cocok untuk data besar
    */

```

```

        * - Kekurangan:
        * Pemakaian memori cukup besar
        */
    }
}

public class Soal2 {

    /*
    * ANALISA COUNTING SORT:
    * -----
    * - Counting Sort bekerja dengan menghitung frekuensi masing-masing nilai.
    * - Cocok untuk data integer dan rentang nilai yang tidak terlalu besar.
    * - Waktu eksekusi:  $O(n + k)$ 
    * -  $n$  = jumlah data
    * -  $k$  = nilai maksimum
    * - Sangat cepat untuk dataset yang nilainya kecil-menengah.
    * - Pada tugas ini digunakan untuk sorting besar → kecil (descending).
    */

    public static void countingSort(int[] arr) {

        // Mencari nilai terbesar untuk menentukan ukuran array count
        int max = arr[0];
        for (int i : arr)
            if (i > max)
                max = i;

        // Array untuk menyimpan frekuensi kemunculan
        int[] count = new int[max + 1];

        // Menghitung kemunculan setiap nilai
        for (int i : arr)
            count[i]++;

        /*
        * MENYUSUN ULANG DATA (DESCENDING)
        * -----
        * - Iterasi dimulai dari angka terbesar ( $\text{max} \rightarrow 0$ )
        * - Jika  $\text{count}[i] > 0$ , berarti angka itu muncul
        * - Masukkan angka tersebut ke array hasil
        */
        int index = 0;
        for (int i = max; i >= 0; i--) {
            while (count[i] > 0) {
                arr[index++] = i;
                count[i]--;
            }
        }
    }

    public static void printArr(int[] arr) {
        for (int i : arr)

```

```

        System.out.print(i + " ");
        System.out.println();
    }

    public static void main(String[] args) {

        // DATA AWAL
        int[] data = { 42, 17, 8, 60, 23, 91, 15, 50 };

        System.out.println("=== COUNTING SORT DESCENDING ===");
        System.out.print("Data sebelum sorting: ");
        printArr(data);

        countingSort(data);

        System.out.print("Data setelah sorting: ");
        printArr(data);

        /*
         * ANALISA KINERJA COUNTING SORT:
         * -----
         * Waktu:  $O(n + k)$ 
         * - Sangat cepat jika nilai max tidak terlalu besar
         * - Tidak melakukan perbandingan antar elemen (non-comparative sorting)
         * Kelebihan:
         * Super cepat untuk data integer
         * Tidak tergantung urutan awal data
         * Kekurangan:
         * Tidak cocok jika nilai maksimum terlalu besar
         */
    }
}

```

Tautan video:

<https://youtu.be/Q-qcKtlQSDQ>